

MFMA Ordering

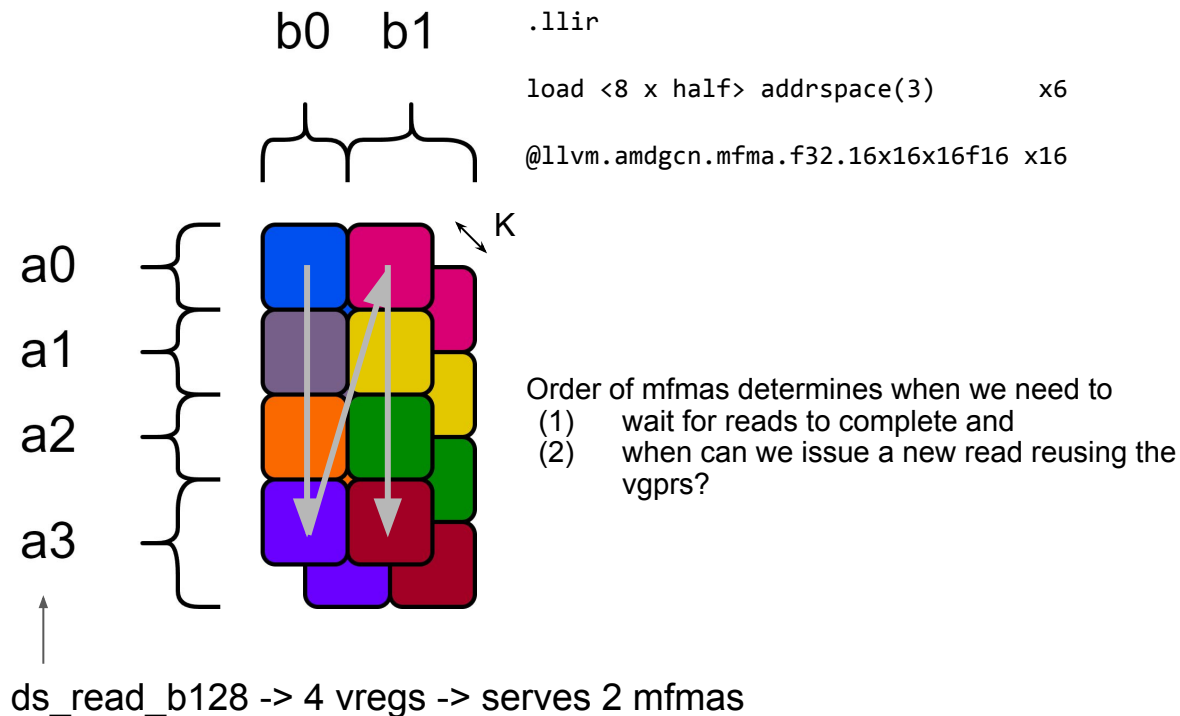
David Tanner
Nov 2024

Overview

- Theory of mfma order ~ register usage.
- Why can't the backend just do it for us?
- Performance results.
- Conclusions and next steps.

Theory of mfma order ~ register usage.

4M x 2N x 2K - Order matters



.llir

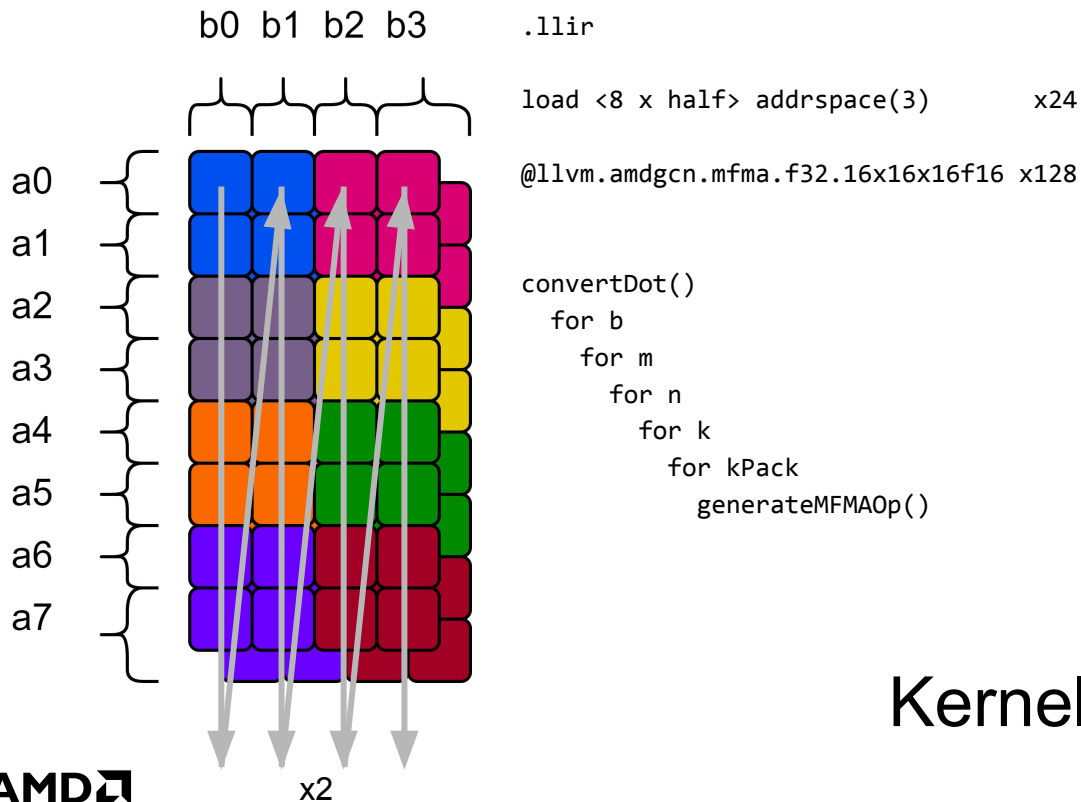
load <8 x half> addrspac(3) x6

@llvm.amdgcn.mfma.f32.16x16x16f16 x16

.amdgcn - column major order

```
ds_read_b128 a0 -> a0k0, a0k1
ds_read_b128 b0 -> b0k0, b0k1
mfma a0k0,b0k0 // wait a0,b0
mfma a0k1,b0k1
ds_read_b128 a1 -> a1k0, a1k1
mfma a1k0,b0k0 // wait a1
mfma a1k1,b0k1
ds_read_b128 a2 -> a2k0, a2k1
mfma a2k0,b0k0 // wait a2
mfma a2k1,b0k1
ds_read_b128 a3 -> a3k0, a3k1
mfma a3k0,b0k0 // wait a3
mfma a3k1,b0k1 // done w/ b0
ds_read_b128 b1 -> b1k0, b1k1
mfma a0k0,b1k0 // wait b1
mfma a0k1,b1k1 // done w/ a0
mfma a1k0,b1k0
mfma a1k1,b1k1 // done w/ a1
mfma a2k0,b1k0
mfma a2k1,b1k1 // done w/ a2
mfma a3k0,b1k0
mfma a3k1,b1k1 // done w/ a3, b1
```

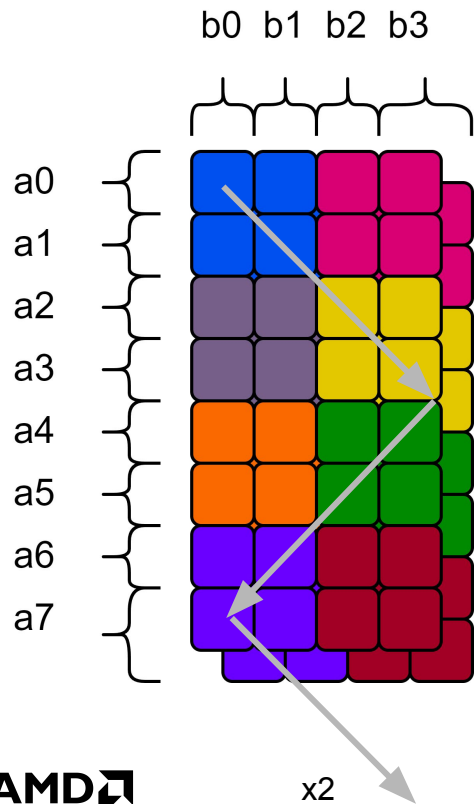
MFMAAs: 8M x 4N x 2k *2 - Current Strategy



Rely on the backend to
schedule / regalloc optimally.

Kernel: 256x256x64 kP2 nS2

Worst Case Order

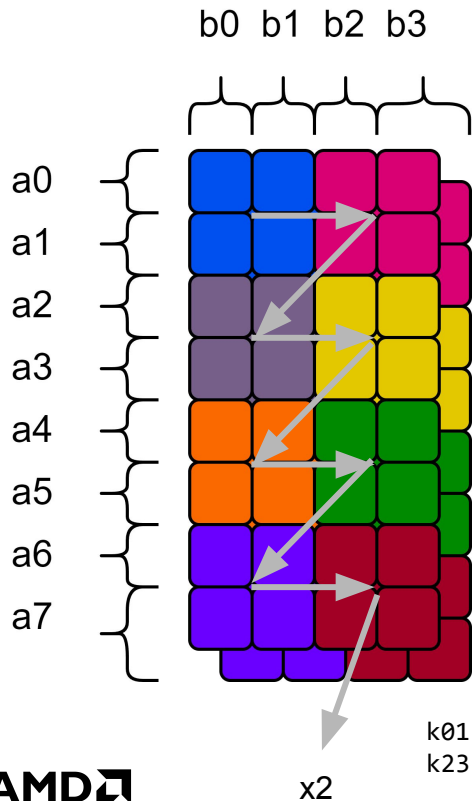


Worst case order would be:

- First 16 mfmas depending on all operands of 24 ds_reads.
- Last 16 mfmas depend on all operands of all 24 ds_reads.
- Therefore 96 vgprs would need to stay alive during middle 96 mfmas.

Backend has universe of bad mfma orders, and heuristics to find a pretty good one.

2x2 Tiling



Tiling aims to:

- (1) Create groups of mfmas which use same ds_read operands.
- (2) Order the groups to minimize vgpr lifetimes (without reloading same data); waiting for ds_reads and retiring vgprs.
- (3) K is outer-most loop b/c independent.

Prefetch at least 8 mfmas at a time.

2x2 column tiling: $6 \times 4 \times 2 = 48$ vregs

2x2 row tiling: $4 \times 4 \times 2 = 32$ vregs

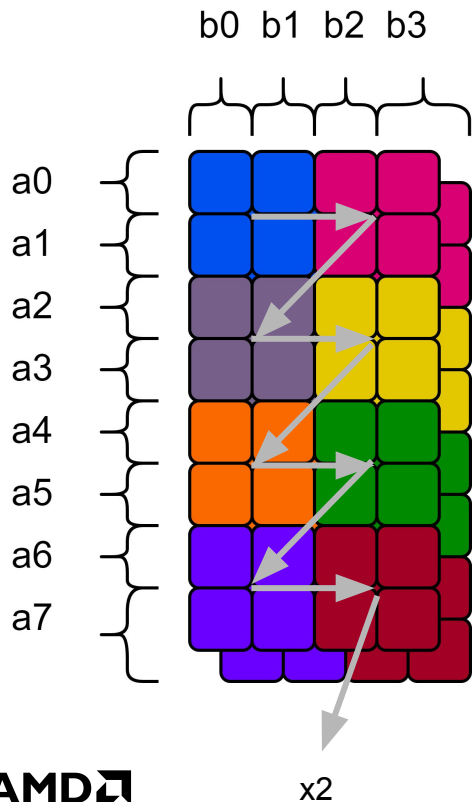
Backend still has flexibility of hoisting ds_reads and using more vgprs.

.amdgc 2x2 Row Tiling

```
ds_read_b128 a0 -> a0k0, a0k1
ds_read_b128 b0 -> b0k0, b0k1
ds_read_b128 a1 -> a1k0, a1k1
ds_read_b128 b1 -> b1k0, b1k1 // 16 vgprs
ds_read_b128 b2 -> b2k0, b2k1
ds_read_b128 b3 -> b3k0, b3k1 // prefetch +8 vgprs
mfma x4 a0k0, b0k0
mfma x4 a0k1, b0k1
mfma x4 a1k0, b0k0
mfma x4 a1k1, b0k1
mfma x4 a0k0, b1k0
mfma x4 a0k1, b1k1
mfma x4 a1k0, b1k0
mfma x4 a1k1, b1k1
ds_read_b128 a2 -> a2k0, a2k1
ds_read_b128 a3 -> a3k0, a3k1 // prefetch +8 vgprs
mfma x4 a0k0, b2k0
mfma x4 a0k1, b2k1
mfma x4 a1k0, b2k0
mfma x4 a1k1, b2k1
mfma x4 a0k0, b3k0
mfma x4 a0k1, b3k1
mfma x4 a1k0, b3k0
mfma x4 a1k1, b3k1 // done with a0, a1 -8 vgprs
ds_read_b128 a4 -> a4k0, a4k1
ds_read_b128 a5 -> a5k0, a5k1 // prefetch +8 vgprs
...
When a67*b01 block is completed, retire a67 and b01;
prefetch a01' and b01'.
When a67*b23 block is completed, retire b23; prefetch b23'.
```

Why can't the compiler just do it for us?

2x2 Tiling



Why doesn't the compiler already do this for us?

1. Scheduler evaluates ops from ready pool based on machine state.
2. Heuristic order:
 - a. Minimize phys-reg live interval length.
 - b. Excel register pressure (spilling).
 - c. Critical reg pressure limits (drops in occupancy).
 - d. Aggressive schedule for latency in loop body.
 - e. Reduce stall cycles for unbuffered resources.
 - f. Cluster edges.
 - g. Minimize reg pressure for non-critical register types.
 - h. Maximize critical resource availability.
 - i. Schedule for latency (ILP).
 - j. **Original node order (from Triton).**
3. There may be no “mathematical” way to get from single-op scheduling to the full structural picture that Triton has (secondary effects may not be examined).
4. Schedules from top and bottom which may conflict in the middle.
5. **Over-indexes on minimizing register-pressure!!!**
 - a. We need to address this; even when registers are available, scheduler unnecessarily delay global_loads.

Performance results.

MFMA Sched.Barrier

Enforce relative order of mfmas only; all other instruction types are allowed to cross barriers.

Sched.barrier adds order dependencies into backend's DAG; only ever 1 mfma in the ready pool at a time.

Metaphor: Triton determines the spine and compiler still chooses all other bones and muscles.

```
int32_t mfmaMask = InstructionKindMask::VALU
| InstructionKindMask::SALU
// | InstructionKindMask::MFMA
| InstructionKindMask::ALL_VMEM
| InstructionKindMask::VMEM_READ
| InstructionKindMask::VMEM_WRITE
| InstructionKindMask::ALL_DS
| InstructionKindMask::DS_READ
| InstructionKindMask::DS_WRITE
| InstructionKindMask::TRANSCEND;
createSchedBarrier(rewriter, loc, mfmaMask);
```

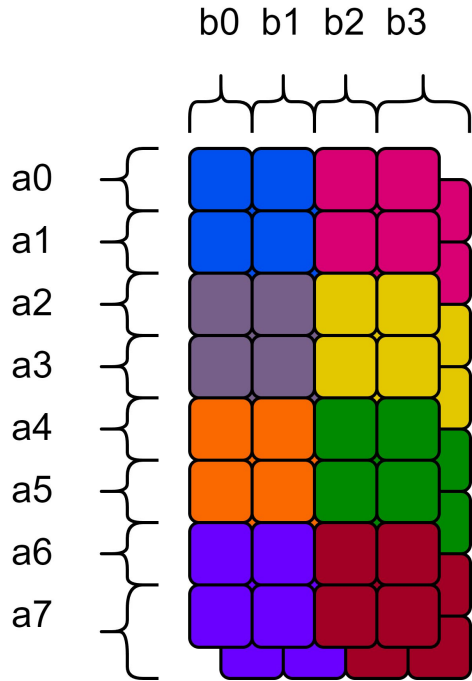
Sched.Group.Barriers (fastest case)

```
// Hoist global loads early.
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::VMEM_READ, 4, group_id);
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::MFMA,      8, group_id);
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::VMEM_READ, 4, group_id);
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::MFMA,     96, group_id);

// Delay ds_writes and interleave.
group_id++;
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::MFMA,     112, group_id);
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::MFMA,      1, group_id);
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::DS_WRITE,   1, group_id); // x8
createSchedGroupBarrier(rewriter, loc, InstructionKindMask::MFMA,      8, group_id);
```

This is not be a scalable strategy.

vgprs & loop cycles for different orders



Order	sched. barriers	vgprs	loop cycles	Comments
main	no	256	5390	
2x2 n,m (col)	no	250	5220	
2x2 m,n (row)	no	250	5410	
2x2 n,m	8 mfmas	234	5083	ds_reads lowered too close to mfmas.
2x2 m,n	8 mfmas	222	5176	ds_reads lowered too close to mfmas. 226 or 230 vgprs would be plenty.
2x2 n,m	2 mfmas	234	4880	
2x2 m,n	2 mfmas	222	5070	ds_reads lowered too close to mfmas.
1x1 m,n	1 mfma	218	5528	ds_reads lowered too close to mfmas.

Conclusions and next steps.

PROs of mfma ordering + sched.barriers(mfma)

- Easy since Triton controls the order in one location.
- Good order may be workload independent; need to benchmark.
- Reduces backend's mistake-space.
- Safe optimization?
 - It may not worsen any current backend heuristics.
 - LLVM should be able to reliably insert order dependencies.
 - Order dependencies only between mfmas should allow compiler flexibility to schedule all other instruction types.

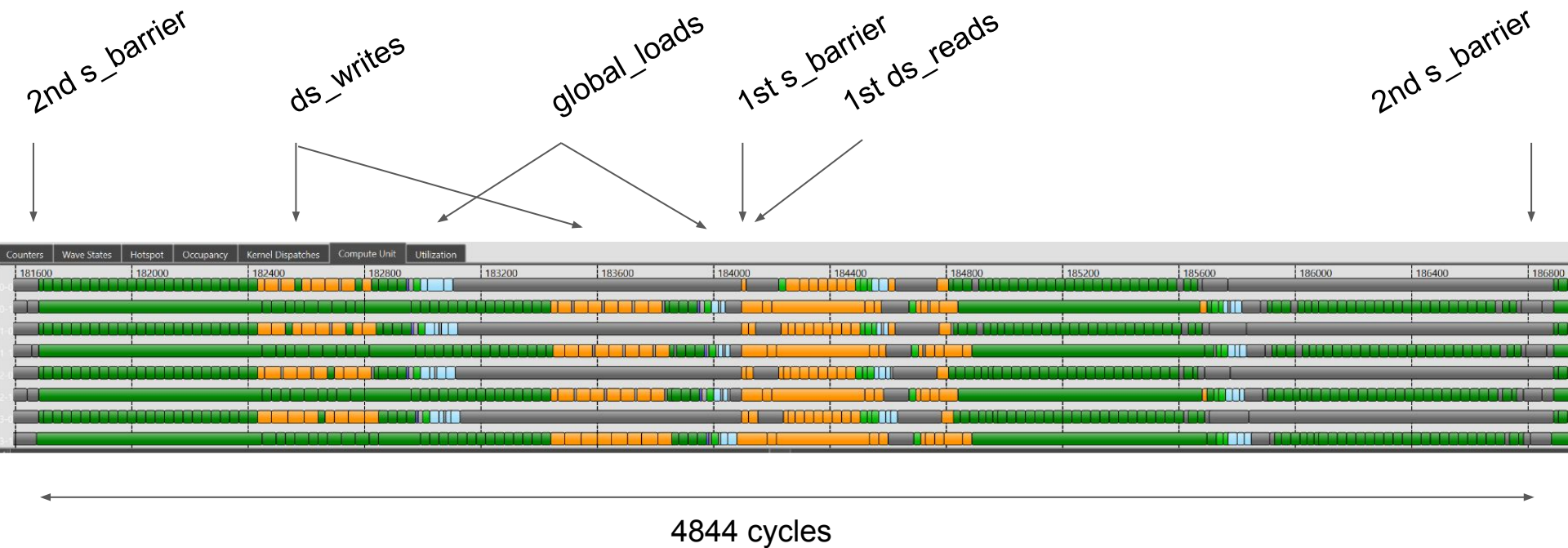
Insufficiencies of mfma ordering + sched.barriers(mfma)

- Backend can still lower ds_reads too close to waitcnt() even when ~30 vgprs are available.
 - Backend would still need more tuning. Maybe iglp_opt?
- Dot slicing could use sched.barriers to keep ds.reads within a block of mfmas.
 - Further restrict compiler to prefetch not too much or too little.

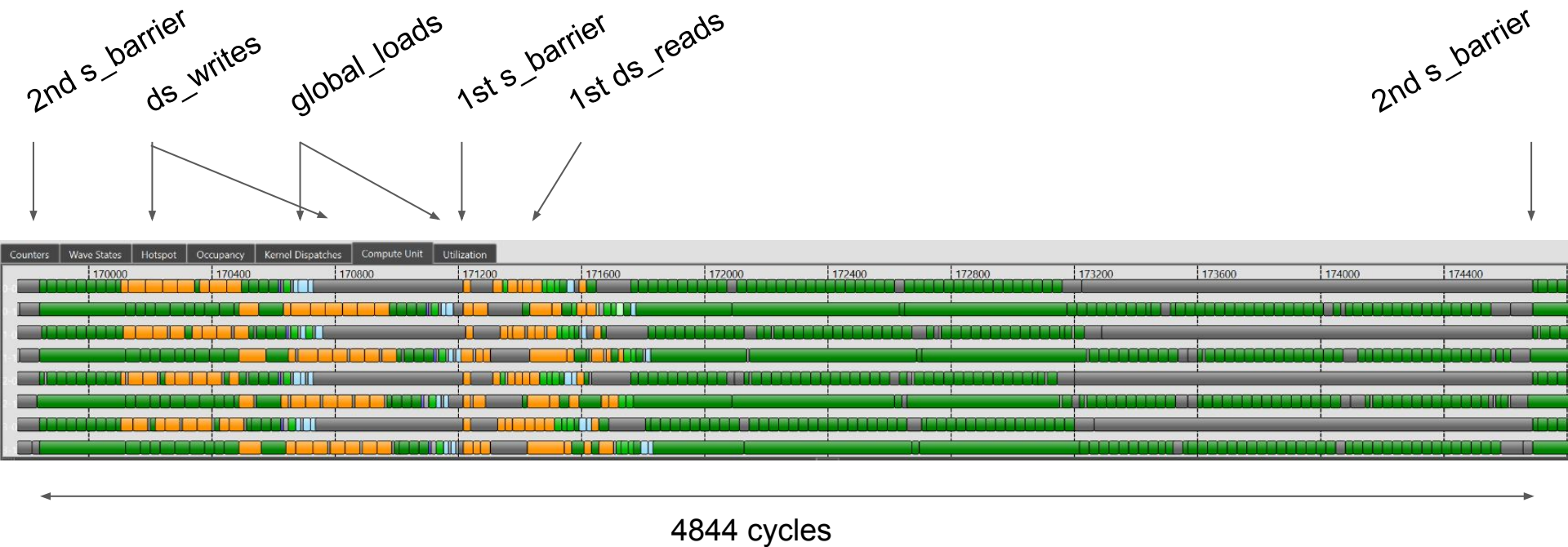
Next Steps

- What tile size to use (based on numRepMnK, kPack, ...)?
 - It could depend on ds_read width?
 - It could be agnostic?
- Should larger of m,n always be outer loop?
 - Smaller dimension as inner loop has “higher latency per register” and should be preferred, but happens to be slower due to backend heuristic.
- It would be more register-efficient to only fetch one K out of LDS at a time, whereas this kernel fetches 2.

Loop of Default Case (256 vgprs, ~5400 cycles)

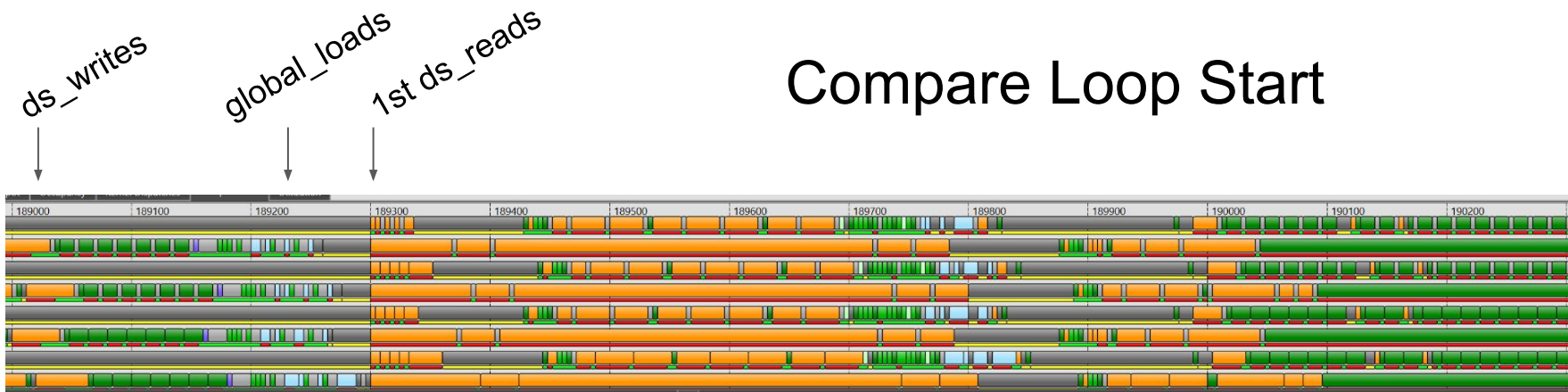


Loop of Fast Case (234 vgprs, 4880 cycles)



Compare Loop Start

Default



2x2 Tiling



ds_reads for tile 0

ds_reads for tile 1

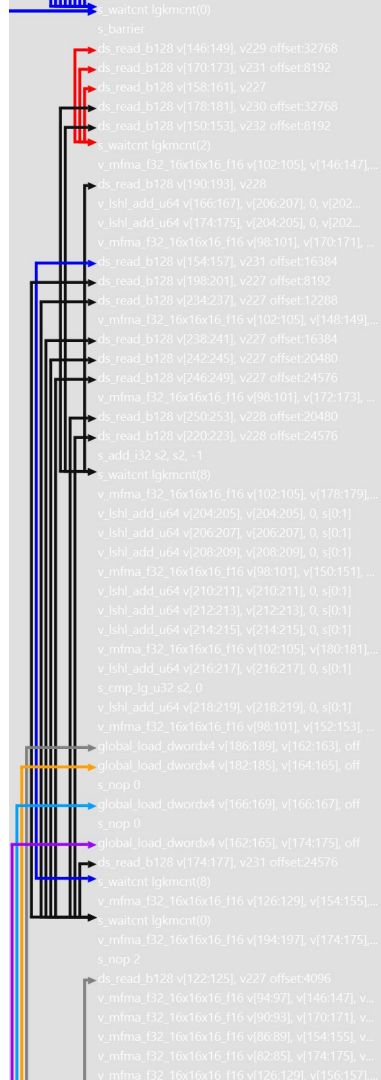
ds_reads for tile 2

1st s_barrier



Default (256 vgprs)

Loop starts interleaving:
16 ds_reads
9 mfmas



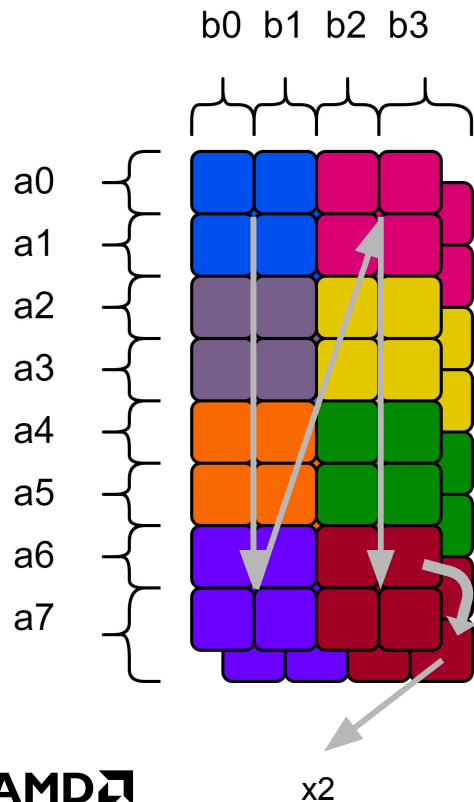
2x2 Tile (234 vgprs)

Loop starts interleaving:
10 ds_reads
12 mfmas



Backup Slides

8M x 4N x 2k *2 (256x256x64 kP2) - tiling 1K



Ideally (for reg minimization) we would like to operate on only 1 K at a time, rather than loading 2 Ks at a time, because the values in one K don't get multiplied by the values in the other K.

1*K at a time.

Prefetch at least 8 mfmas at a time.

2x2 column order: $6 \times 4 = 24$ vregs

2x2 row order: $4 \times 4 = 16$ vregs

k0 [0:15]
k1 [16:31]
k2 [32:47]
k3 [48:65]

Prefer to ds_read_b64 to only get 1 K iter at a time and save registers, where needed.

ds_writes

A lot of time is lost here even when interleaved with mfmas. Need ping-pong to fix this?

